

Module 1: Introduction and Essentials of Python

Learning Objectives

- Introduction to Python programming
- Basic Python syntax and data types
- Variables, operators, and expressions
- Control flow (if/else, loops)
- Functions and modules

Topics Covered

1. Python installation and setup
2. Python interpreter and IDEs
3. Basic data types (int, float, string, boolean)
4. Variables and assignment
5. Arithmetic and comparison operators
6. Conditional statements
7. Loops (for and while)
8. Functions definition and calling
9. Importing modules
10. Basic error handling

Exercises

- Practice problems will be provided throughout the module

1. Python Installation and Setup

```
In [11]: # Check Python version
import sys
print(f"Python version: {sys.version}")
print(f"Python executable: {sys.executable}")
```

Python version: 3.11.13 (main, Jun 5 2025, 08:14:07) [Clang 14.0.6]
Python executable: /opt/miniconda3/envs/python-course/bin/python

2. Basic Data Types

```
In [3]: # Integer
age = 25
print(f"Age: {age}, Type: {type(age)}")
```

```

# Float
height = 5.9
print(f"Height: {height}, Type: {type(height)}")

# String
name = "Alice"
print(f"Name: {name}, Type: {type(name)}")

# Boolean
is_student = True
print(f"Is student: {is_student}, Type: {type(is_student)}")

# None
data = None
print(f"Data: {data}, Type: {type(data)}")

```

Age: 25, Type: <class 'int'>
 Height: 5.9, Type: <class 'float'>
 Name: Alice, Type: <class 'str'>
 Is student: True, Type: <class 'bool'>
 Data: None, Type: <class 'NoneType'>

3. Variables and Assignment

```

In [4]: # Variable assignment
x = 10
y = 20

# Multiple assignment
a, b, c = 1, 2, 3
print(f"a={a}, b={b}, c={c}")

# Swapping variables
x, y = y, x
print(f"After swap: x={x}, y={y}")

# Augmented assignment
count = 5
count += 3 # Same as count = count + 3
print(f"Count after += 3: {count}")

# String formatting
first_name = "John"
last_name = "Doe"
full_name = f"{first_name} {last_name}"
print(f"Full name: {full_name}")

```

a=1, b=2, c=3
 After swap: x=20, y=10
 Count after += 3: 8
 Full name: John Doe

4. Arithmetic and Comparison Operators

```

In [5]: # Arithmetic operators
a, b = 10, 3

print(f"Addition: {a} + {b} = {a + b}")

```

```

print(f"Subtraction: {a} - {b} = {a - b}")
print(f"Multiplication: {a} * {b} = {a * b}")
print(f"Division: {a} / {b} = {a / b}")
print(f"Floor division: {a} // {b} = {a // b}")
print(f"Modulus: {a} % {b} = {a % b}")
print(f"Exponentiation: {a} ** {b} = {a ** b}")

# Comparison operators
x, y = 5, 8
print(f"\nComparison operators:")
print(f"{x} == {y}: {x == y}")
print(f"{x} != {y}: {x != y}")
print(f"{x} < {y}: {x < y}")
print(f"{x} > {y}: {x > y}")
print(f"{x} <= {y}: {x <= y}")
print(f"{x} >= {y}: {x >= y}")

# Logical operators
p, q = True, False
print(f"\nLogical operators:")
print(f"p and q: {p and q}")
print(f"p or q: {p or q}")
print(f"not p: {not p}")

```

Addition: $10 + 3 = 13$
 Subtraction: $10 - 3 = 7$
 Multiplication: $10 * 3 = 30$
 Division: $10 / 3 = 3.3333333333333335$
 Floor division: $10 // 3 = 3$
 Modulus: $10 \% 3 = 1$
 Exponentiation: $10 ** 3 = 1000$

Comparison operators:

$5 == 8$: False
 $5 != 8$: True
 $5 < 8$: True
 $5 > 8$: False
 $5 <= 8$: True
 $5 >= 8$: False

Logical operators:

p and q : False
 p or q : True
 not p : False

5. Conditional Statements

In [6]: `# if-else statements`
`score = 85`

```

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:

```

```

    grade = "F"

print(f"Score: {score}, Grade: {grade}")

# Ternary operator
age = 20
status = "adult" if age >= 18 else "minor"
print(f"Age: {age}, Status: {status}")

# Nested conditions
temperature = 25
is_sunny = True

if temperature > 20:
    if is_sunny:
        print("Great weather for outdoor activities!")
    else:
        print("Warm but cloudy day")
else:
    print("It's a bit cold today")

```

Score: 85, Grade: B
 Age: 20, Status: adult
 Great weather for outdoor activities!

6. Loops

```

In [7]: # for loop
fruits = ["apple", "banana", "orange", "grape"]
print("Fruits:")
for fruit in fruits:
    print(f"- {fruit}")

# for loop with range
print("\nNumbers 1 to 5:")
for i in range(1, 6):
    print(f"Number: {i}")

# while loop
count = 0
print("\nWhile loop countdown:")
while count < 5:
    print(f"Count: {count}")
    count += 1

# break and continue
print("\nBreak and continue example:")
for num in range(1, 11):
    if num == 3:
        continue # Skip 3
    if num == 8:
        break # Stop at 8
    print(f"Number: {num}")

# enumerate
print("\nEnumerate example:")
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

```

Fruits:
– apple
– banana
– orange
– grape

Numbers 1 to 5:
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5

While loop countdown:
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4

Break and continue example:
Number: 1
Number: 2
Number: 4
Number: 5
Number: 6
Number: 7

Enumerate example:
0: apple
1: banana
2: orange
3: grape

7. Functions

```
In [8]: # Basic function
def greet(name):
    return f"Hello, {name}!"

print(greet("Alice"))

# Function with multiple parameters
def calculate_area(length, width):
    return length * width

area = calculate_area(5, 3)
print(f"Area: {area}")

# Function with default parameters
def greet_with_title(name, title="Mr."):
    return f"Hello, {title} {name}!"

print(greet_with_title("Smith"))
print(greet_with_title("Johnson", "Dr.))

# Function with keyword arguments
def create_profile(name, age, city="Unknown", country="Unknown"):
    return f"Name: {name}, Age: {age}, City: {city}, Country: {country}"
```

```

profile = create_profile("Bob", 30, country="USA")
print(profile)

# Function returning multiple values
def get_name_parts(full_name):
    parts = full_name.split()
    return parts[0], parts[-1] if len(parts) > 1 else ""

first, last = get_name_parts("John Doe")
print(f"First: {first}, Last: {last}")

# Lambda function
square = lambda x: x ** 2
print(f"Square of 5: {square(5)}")

# Using lambda with built-in functions
numbers = [1, 2, 3, 4, 5]
squared_numbers = list(map(lambda x: x ** 2, numbers))
print(f"Squared numbers: {squared_numbers}")

```

```

Hello, Alice!
Area: 15
Hello, Mr. Smith!
Hello, Dr. Johnson!
Name: Bob, Age: 30, City: Unknown, Country: USA
First: John, Last: Doe
Square of 5: 25
Squared numbers: [1, 4, 9, 16, 25]

```

8. Importing Modules

```

In [9]: # Import entire module
import math
print(f"Square root of 16: {math.sqrt(16)}")
print(f"Value of pi: {math.pi}")

# Import specific functions
from datetime import datetime, date
now = datetime.now()
today = date.today()
print(f"Current time: {now}")
print(f"Today's date: {today}")

# Import with alias
import random as rnd
random_number = rnd.randint(1, 10)
print(f"Random number: {random_number}")

# Import all from module (not recommended for large modules)
from math import *
print(f"Factorial of 5: {factorial(5)}")

# Check available functions in a module
print(f"\nMath module functions: {dir(math)[:10]}...") # First 10 functi

```

Square root of 16: 4.0
Value of pi: 3.141592653589793
Current time: 2025-09-07 21:28:24.335830
Today's date: 2025-09-07
Random number: 3
Factorial of 5: 120

Math module functions: ['__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh']...

9. Basic Error Handling

```
In [10]: # Basic try-except
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        return "Cannot divide by zero!"
    except TypeError:
        return "Invalid input types!"

print(safe_divide(10, 2))
print(safe_divide(10, 0))
print(safe_divide(10, "2"))

# Multiple exception types
def get_list_element(lst, index):
    try:
        return lst[index]
    except IndexError:
        return "Index out of range!"
    except TypeError:
        return "Invalid index type!"

numbers = [1, 2, 3, 4, 5]
print(get_list_element(numbers, 2))
print(get_list_element(numbers, 10))
print(get_list_element(numbers, "2"))

# try-except-else-finally
def process_file(filename):
    try:
        file = open(filename, 'r')
        content = file.read()
    except FileNotFoundError:
        print(f"File {filename} not found!")
        return None
    except PermissionError:
        print(f"Permission denied to read {filename}")
        return None
    else:
        print("File read successfully!")
        return content
    finally:
        print("File processing completed")
```

```
# This will show the error handling  
result = process_file("nonexistent.txt")
```

```
5.0  
Cannot divide by zero!  
Invalid input types!  
3  
Index out of range!  
Invalid index type!  
File nonexistent.txt not found!  
File processing completed
```

In []: